CHAPITRE 2:

HADOOP

BIG DATA



Ons Ben Rejeb
ISET de Zaghouan
Département TI
Mastère ISID

Plan

- Hadoop
- Hadoop, HDFS et Map Reduce
- Eco système de Hadoop
- HDFS: Hadoop Distributed File System
- Quelques commandes HDFS
- Map-Reduce
- Map-Reduce : Exemple
- Map-Reduce : Fonctionnement
- Démon de Map-Reduce

Plan

- Hadoop Design Patterns
- Patrons de filtrage
- Patrons de récapitulation
- Patrons Structurels
- Atouts de Hadoop
- Inconvénients de Hadoop
- Exercice

Hadoop

- Le projet Hadoop consiste en deux grandes parties:
  - Stockage des données : **HDFS** (*Hadoop **D**istributed **F**ile **S**ystem*)
  - Traitement des données : **MapReduce**



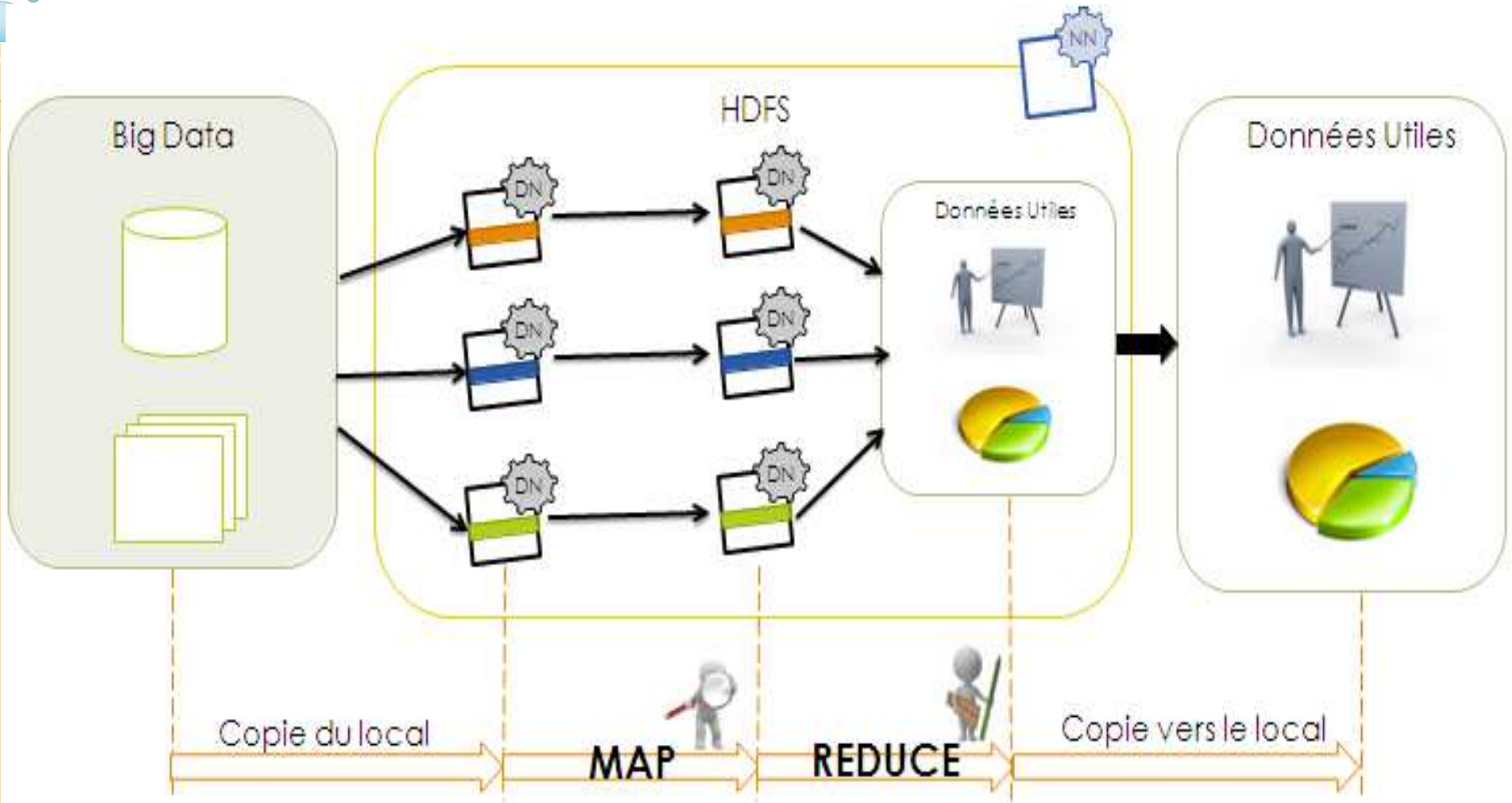
Hadoop

- Principe :
  - Diviser les données
  - Les sauvegarder sur une collection de machines, appelées *cluster*
  - Traiter les données directement là où elles sont stockées, plutôt que de les copier à partir d'un serveur distribué
- Il est possible d'ajouter des machines à votre cluster

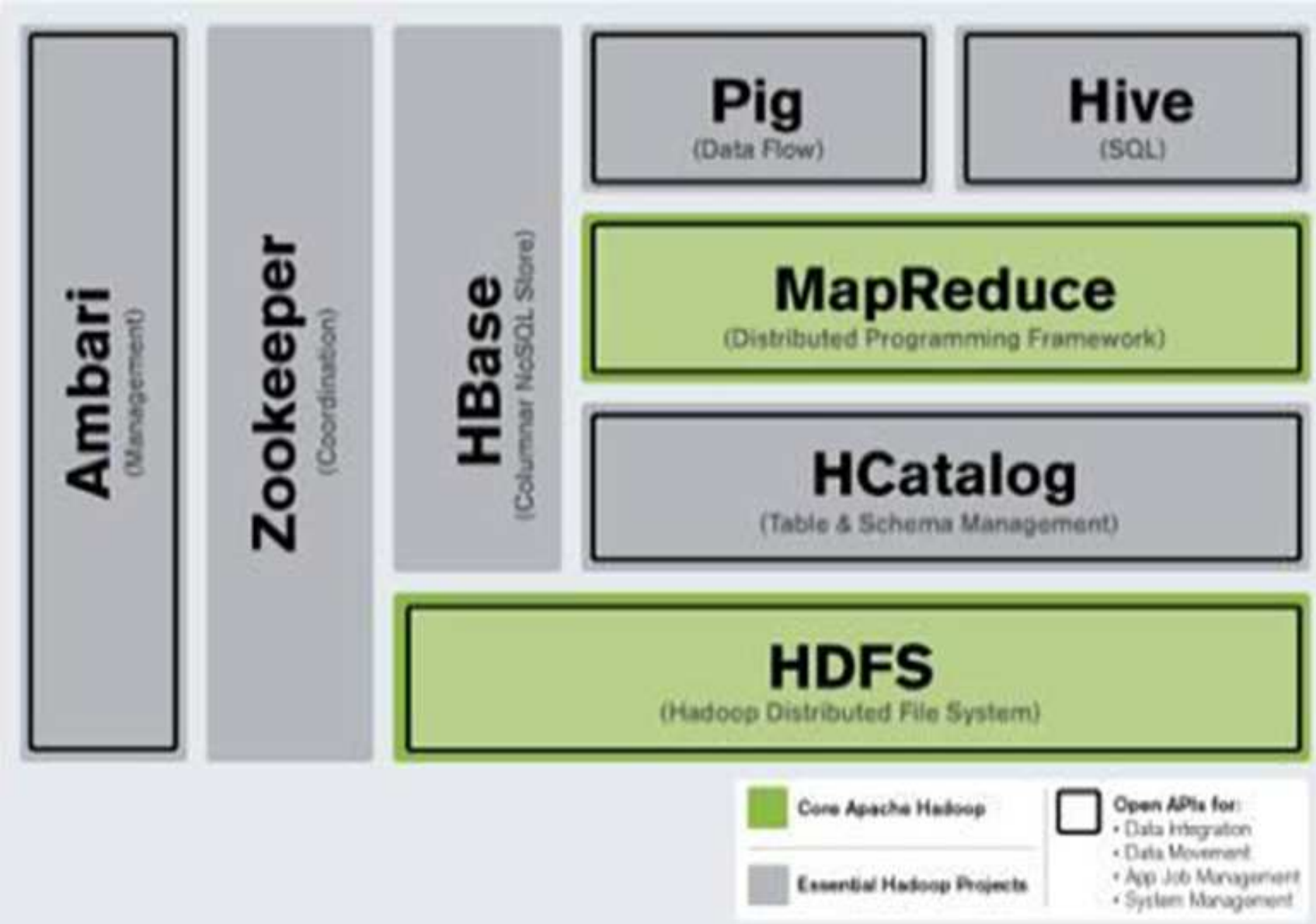
Hadoop

- Hadoop est un framework Java qui permet de constituer une plateforme Big Data complète
  - Adaptable sur des très gros volumes
  - Tolérant aux pannes
  - Open source
  - Hardware « banalisé »

Hadoop, HDFS et Map Reduce



Eco système de Hadoop



Eco système de Hadoop

- Hcatalog: Couche de gestion des métadonnées
- Pig : script simple orienté traitement par flux (data flow)
- Hive : manipuler les données avec des requêtes SQL
- Hbase : bases de données temps réel (NoSQL)
- Zookeeper : coordination pour la fiabilité de distribution
- Ambari : Gestion et monitoring des clusters de Hadoop

HDFS: Hadoop Distributed File System

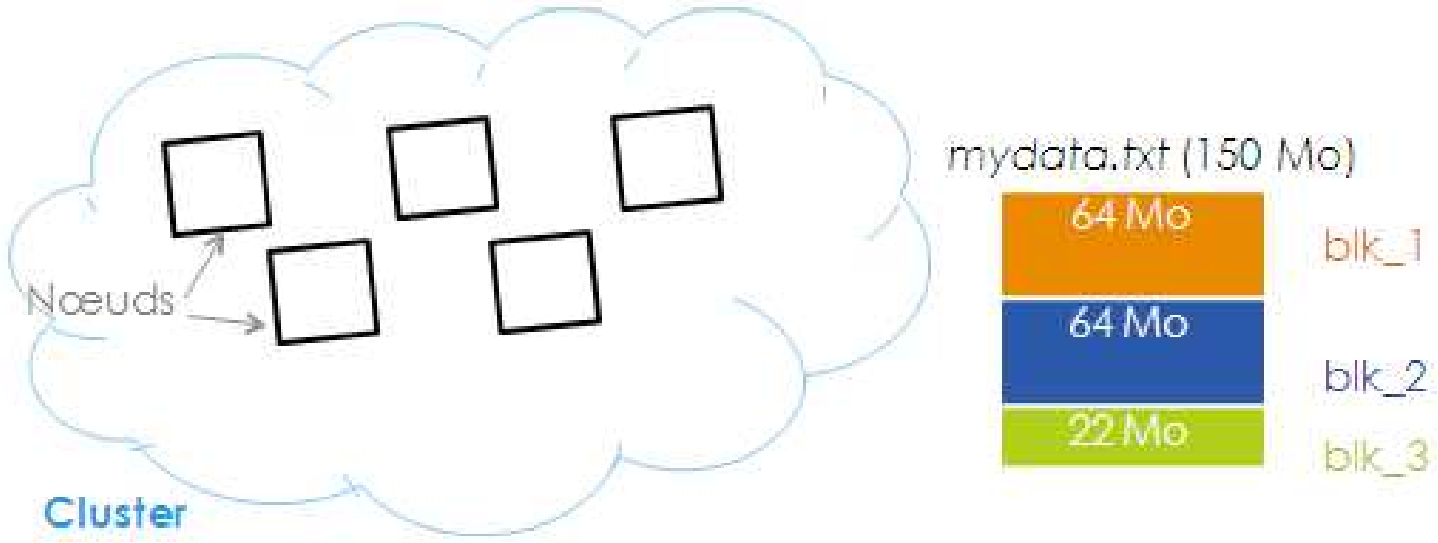
- HDFS est un système de fichiers distribué, extensible et portable
- Permet de stocker de très gros volumes de données sur un grand nombre de machines (nœuds) équipées de disques durs banalisés

➲ Cluster

- Garantie de la cohérence des données

HDFS: Hadoop Distributed File System

- Un fichier mydata.txt enregistré dans HDFS est décomposé en grands blocs (par défaut 64Mo), chaque bloc ayant un nom unique: blk_1, blk_2…



HDFS: Hadoop Distributed File System

- Chaque bloc est enregistré dans un nœud différent du cluster
- **DataNode** : *démon* sur chaque nœud du cluster
- **NameNode** :
  - Démon s'exécutant sur une machine séparée
  - Contient des *méta-données*
  - Permet de retrouver les nœuds qui exécutent les blocs d'un fichier

HDFS: Hadoop Distributed File System

- Quels sont les problèmes possibles?
  - ☑ Panne de réseau ?
  - ☑ Panne de disque sur les DataNodes ?
  - ☐ Pas tous les DN sont utilisés ?
  - ☐ Les tailles des blocks sont différentes ?
  - ☑ Panne de disque sur les NameNodes ?

HDFS: Hadoop Distributed File System

- Le problème des données perdues ne se pose pas puisque:
  - Hadoop réplique chaque bloc 3 fois
  - Il choisit 3 nœuds au hasard, et place une copie du bloc dans chacun d'eux
  - Si le nœud est en panne, le NN le détecte, et s'occupe de répliquer encore les blocs qui y étaient hébergés pour avoir toujours 3 copies stockées
  - ➲ Concept de **Rack Awareness** (rack = baie de stockage)

HDFS: Hadoop Distributed File System

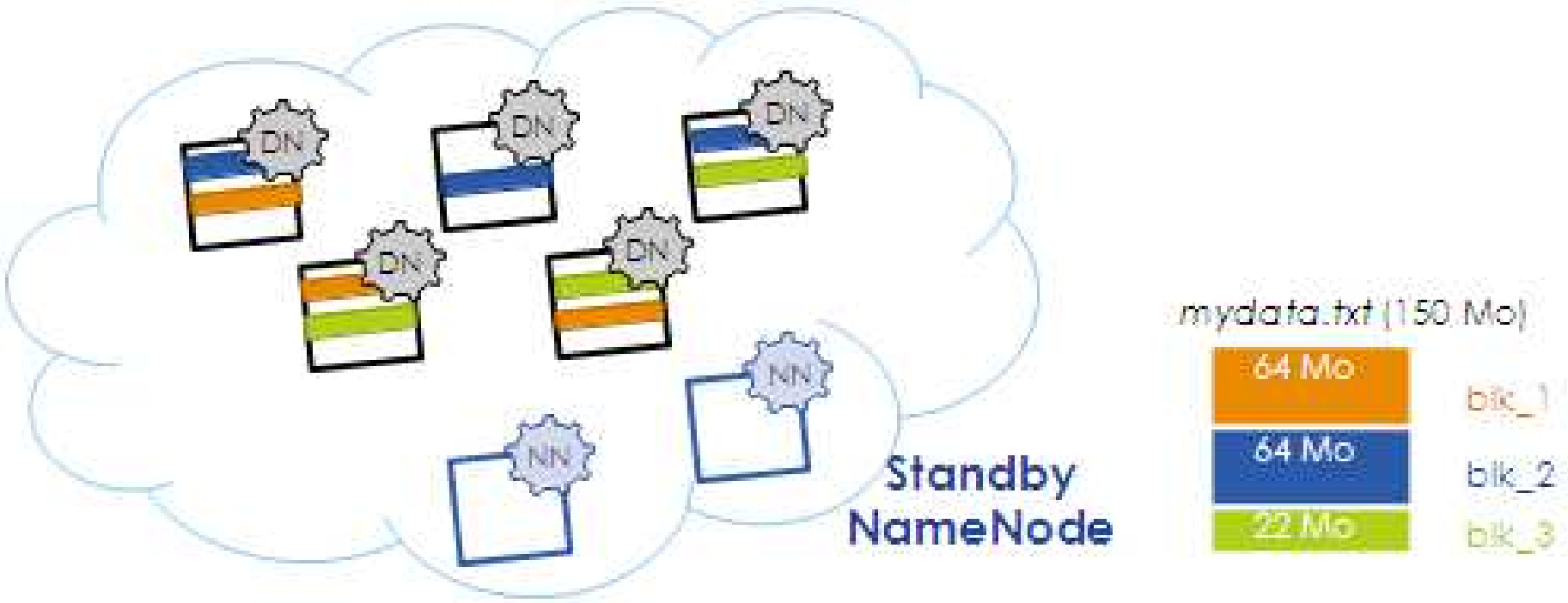
- Si le NameNode a un problème :
  - ☐ Données inaccessibles
  - ☑ Données perdues à jamais
  - ☑ Pas de problème
- Si c'est un problème d'accès (réseau), les données sont temporairement inaccessibles
- Si le disque du NN est défaillant, les données seront perdues à jamais

HDFS: Hadoop Distributed File System

- Pour éviter cela
  - Le NameNode sera dupliqué, non seulement sur son propre disque, mais également quelque part sur le système de fichier du réseau
  - Définition d'un autre NN (standby namenode) pour reprendre le travail si le NameNode actif est défaillant

HDFS: Hadoop Distributed File System

- Répartition, duplication et disponibilité



Quelques commandes HDFS

-cat : affiche le contenu d'un fichier

-text : comme cat mais sait afficher des données compressées

-chgrp, -chmod, -chown : modification des permissions

-put, -get, -copyFromLocal, -copyToLocal : import / export entre le système de fichier local et HDFS

Quelques commandes HDFS

-ls, -lsr : liste les fichiers / répertoires

-mv, -moveFromLocal, -moveToLocal :

déplace les fichiers

-stat : informations statistiques sur les ressources

(taille des blocs, nombre de blocs, type de fichiers,

etc.)

Quelques commandes HDFS

$ hadoop fs –ls /user/brian/

$ hadoop fs -lsr

$ hadoop fs –mkdir notes

$ hadoop fs –put ./training/commands.txt notes

$ hadoop fs –chmod 777 notes/commands.txt

$ hadoop fs –cat notes/commands.txt | more

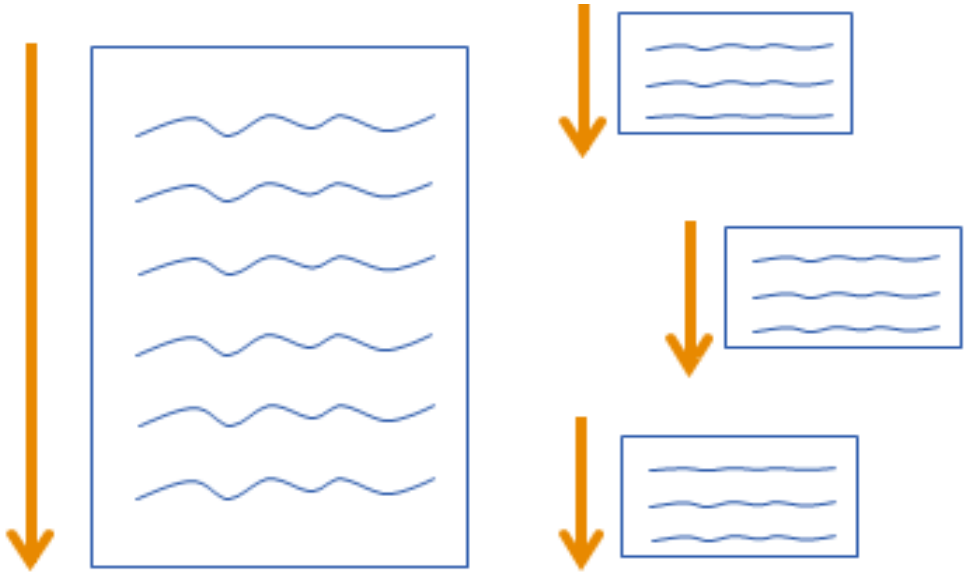
$ hadoop fs –rm notes/*.txt

Map-Reduce

- Patron d'architecture de développement permettant de traiter des données volumineuses de manière parallèle et distribuée
- A la base, le langage Java est utilisé, mais grâce à une caractéristique de Hadoop appelée *Hadoop Streaming,* il est possible d'utiliser d'autres langages comme Python ou Ruby

Map-Reduce

- Au lieu de parcourir le fichier séquentiellement (bcp de temps), il est divisé en morceaux qui sont parcourus en parallèle.



Map-Reduce : Exemple

- Imaginons que vous ayez plusieurs magasins que vous gérez à travers le monde
- Un très grand livre de comptes contenant TOUTES les ventes
- **Objectif** : Calculer le total des ventes par magasin pour l'année en cours
- Supposons que les lignes du livres aient la forme suivante : Jour Ville Produit Prix

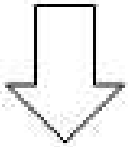
Map-Reduce : Exemple

- **Traitement séquentiel:**
  - Pour chaque entrée, saisir la ville et le prix de vente
  - Si on trouve une entrée avec une ville déjà saisie, on les regroupe en faisant la somme des ventes
  - Calcul traditionnel (Hashtables, sous forme de Clef, Valeur) (adresse, total des ventes)

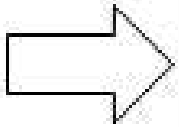
Map-Reduce : Exemple



2012-01-01	London	Clothes	25.99
2012-01-01	Miami	Music	12.15
2012-01-02	NYC	Toys	3.10
2012-01-02	Miami	Clothes	50.00



London	25.99
Miami	12.15
NYC	3.10



London	25.99
Miami	62.15
NYC	3.10

Map-Reduce : Exemple

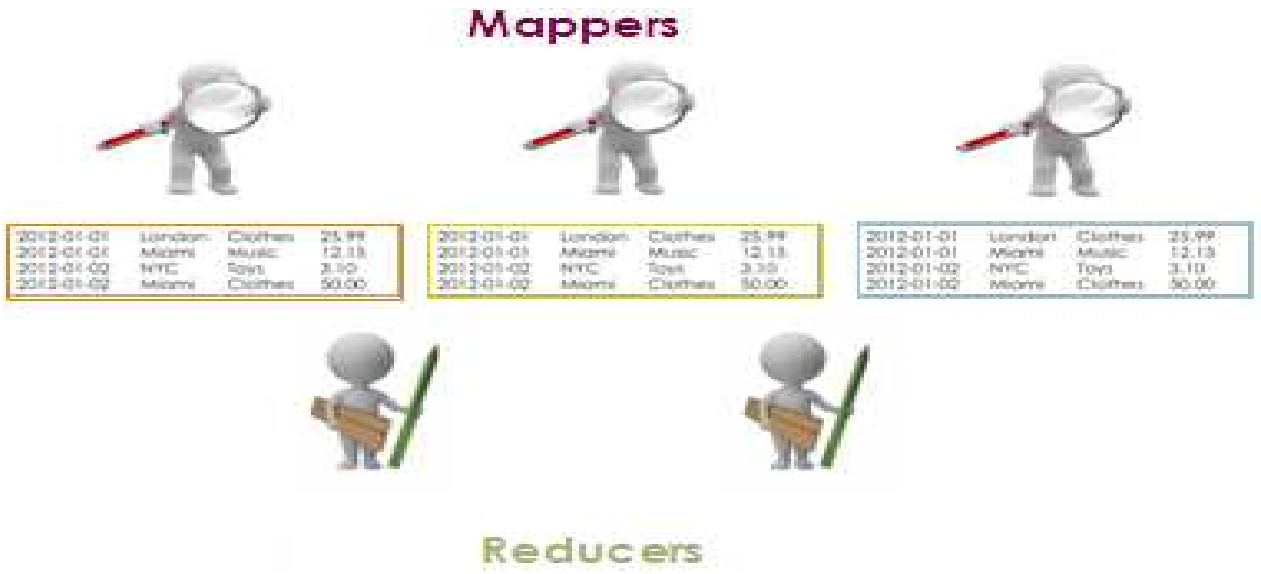
- Si on utilise les hashtables sur 1 To, Problèmes?
  - ☐ Ca ne marche pas?
  - ☑ Problème de mémoire?
  - ☑ Temps de traitement long?
  - ☐ Réponses erronées?
- Plus on a de magasins, plus l'ajout des valeurs à la table est long
- Il est possible de tomber à court de mémoire si la table est taille importante

Map-Reduce : Exemple

- **Map-Reduce (solution parallèle):**
  - Moyen plus efficace et rapide pour traiter les données
  - Au lieu d'avoir une seule personne qui parcourt le livre, si on en recrutait plusieurs?
  - Appeler un premier groupe les *Mappers* et un autre les *Reducers*

Map-Reduce : Exemple

- Diviser le livre en plusieurs parties, et en donner une à chaque *Mapper*
- Les Mappers peuvent travailler en même temps, chacun sur une partie des données



Map-Reduce : Exemple

- **Mappers :**
  - Pour chaque entrée, saisir la ville, et le total des ventes et les enregistrer dans une fiche
  - Rassembler les fiches du même magasin dans une même pile
  - On peut ne pas trouver toute les villes dans chacun des mappers

Map-Reduce : Exemple

- **Reducers :**
  - Chaque Reducer sera responsable d'un ensemble de magasins
  - Ils collectent les fiches qui leur sont associées des différents Mappers
  - Ils regroupent les petites piles d'une même ville en une seule

Map-Reduce : Exemple

- Ils parcourent ensuite chaque pile par ordre alphabétique des villes (L.A avant Miami), et font la somme de l'ensemble des enregistrements



Map-Reduce : Fonctionnement

- Les Mappers sont de petits programmes qui commencent par traiter chacun une petite partie des données
- Ils fonctionnent en parallèle
- Leurs sorties représentent les *enregistrements intermédiaires:* sous forme d'un couple (*clef, valeur*)

Map-Reduce : Fonctionnement

- Une étape de *Mélange et Tri* s'ensuit
  - *Mélange* : Sélection des piles de fiches à partir des Mappers
  - *Tri* : Rangement des piles par ordre au niveau de chaque Reducer
- Chaque Reducer traite un ensemble d'enregistrements à la fois, pour générer les résultats finaux

Map-Reduce : Fonctionnement

- Pour avoir **un résultat trié** par ordre, on doit :
  - Soit avoir un seul Reducer, mais ça ne se met pas bien à l'échelle
  - Soit ajouter une autre étape permettant de faire le tri final
- Si on a plusieurs Reducers, on ne peut pas savoir lesquels traitent quelles clefs : le partitionnement est aléatoire.

Démon de Map-Reduce

- JobTracker
  - Divise le travail sur les Mappers et Reducers, s'exécutant sur les différents nœuds
- TaskTracker
  - S'exécute sur chacun des nœuds pour exécuter les vraies tâches de Map-Reduce

Démon de Map-Reduce

- Choisit en général de traiter (Map ou Reduce) un bloc sur la même machine que lui
- S'il est déjà occupé, la tâche revient à un autre tracker, qui utilisera le réseau (rare)



Hadoop Design Patterns

- Les designs patterns (Patrons de Conception) représentent les types de traitements les plus utilisés avec Hadoop
- Classés en trois catégories:
  - Patrons de Filtrage
    - Echantillonnage de données
    - Listes des top-n

Hadoop Design Patterns

- Patrons de Récapitulation
  - Comptage des enregistrement
  - Trouver les min et les max
  - Statistiques
  - Indexes
- Patrons Structurels
  - Combinaison de données relationnelles

Patrons de filtrage

- Ne modifient pas les données
- Trient, parmi les données présentes, lesquelles garder et lesquelles enlever
- On peut obtenir:
  - **Des filtres simples** : définition d'une fonction indiquant le critère de filtrage

Patrons de filtrage

- **L'échantillonnage** : création d'un petit ensemble d'enregistrements à partir d'un grand ensemble, en retenant des échantillons (la valeur la plus élevée d'un champs particulier)
- **L'échantillonnage aléatoire** : retenir un échantillon représentatif des données initiales
- **Les listes Top-n**

Patrons de récapitulation

- Permettent de vous donner une idée de haut niveau de vos données
- On distingue deux types:
  - **Index** : Tels que les index à la fin d'un livre, ou les index utilisés par google pour représenter les pages web

Patrons de récapitulation

- **Récapitulation (ou résumé) numérique** :

  par exemple:

  - Chercher des chiffres, des comptes (combien dispose-t-on d'un certain type d'entrées)
  - Min et Max
  - Premier et dernier
  - Moyenne
  - Etc.

Patrons de récapitulation

- Mélangeur:
  - Combiner entre les mappers et les reducers
  - Réaliser la réduction en local dans chacun des DataNodes avant de faire appel au nœud réducteur principal
- **Exemple** : Calculer la moyenne des ventes par jour de la semaine

Patrons de récapitulation

- Sans combiner
  - Les Mappers parcourent les données, et affichent le couple *(Jour_de_la_semaine, montant)*
  - Pour chaque jour, le Reducer conserve une somme et un compteur
  - Il divise à la fin la somme par le compteur

Patrons de récapitulation

- Avec combiner
  - Chaque nœud réalise une première réduction où les moyennes locales sont calculées
  - Le Reducer final regroupe ces moyennes et synthétise la moyenne finale
  - Nombre d'enregistrements envoyés au réducteur significativement réduit
  - Temps nécessaire pour la réduction diminue

Patrons Structurels

- Utilisés quand les données proviennent d'une base de données structurée
- Plusieurs tables, donc plusieurs sources de données, liées par clef étrangère
- Les données des différentes tables sont exportées sous forme de fichiers délimités

Atouts de Hadoop

- La gestion des défaillances (Forte tolérance aux pannes)
- La sécurité et persistance des données
- La montée en charge
- La complexité réduite
- Le coût réduit

Inconvénients de Hadoop

- Difficulté d'intégration avec d'autres systèmes informatiques
- Administration complexe
- Traitement de données différé et temps de latence important
- Produit en développement continu

Exercice

- Résumé des métadonnées figurant dans le cluster :

Fichier	Bloc	Nœud	Taille
F1	B1	DN1	
F1	B2	DN2	
F1	B3	DN3	24Mo
F1	B1	DN2	
F1	B2	DN3	
F1	B3	DN1	24MO
F1	B1	DN3	
F1	B2	DN1	
F1	B3	DN2	24Mo
F2	B4	DN1	
F2	B5	DN2	10MO
F2	B4	DN4	
F2	B5	DN5	10MO
F2	B4	DN5	
F2	B5	DN4	10MO

Exercice

Questions :

1. Quelle est la taille de chaque fichier sachant que la taille indiquée dans le tableau est la taille du dernier bloc de chaque fichier ? Justifier votre réponse.
2. Donner dans un schéma la répartition des différents blocs sur les différents *data node* du *cluster* ?

Exercice

3. Quelle est la taille occupée par les blocs dans chaque nœud ? Justifier votre réponse.
4. Dans le cas où le disque dur de l'un des *data node* tombe en panne, quelle est la réaction du système *Hadoop*?

Exercice

5. Supposons que le disque dur qui contient le *data node* DN2 tombe en panne, donner le contenu du tableau relatif aux métadonnées ainsi que la répartition des blocs dans le *cluster* sous forme de schéma après la réaction du système *Hadoop*?